

机器学习Project2

≡ 姓名 周瑞奇 电子工程系 2025210719

一、引言

本项目基于OGBN的arxiv、proteins和product数据集实现并测试了四种基线算法：MLP、LPA、GCN和大语言模型（LLM）。具体来说，我们的工作如下：

数据预处理

我们手动下载了OGBN-arxiv，OGBN-proteins和OGBN-product数据集，并按照需求解压了对应的节点特征文件、边信息文件、数据集划分文件等。针对OGBN-proteins数据集，由于其本身并没有提供节点特征信息，我们遵循参考文献【1】中的做法，将每个节点的入边的特征平均值作为该节点的特征向量。针对OGBN-arxiv，我们还额外下载了对应的文章的标题-摘要文件。考虑到原始OGBN-arxiv数据集规模庞大，在使用LLM对OGBN-arxiv数据进行预测时，我们仅从训练集中抽出8000条作为新的训练数据，而从测试集中抽出500条作为新的测试集。

算法实现

- **MLP**：仅使用单个节点特征作为输入向量，而不考虑整体的图结构。
- **LPA**：仅使用节点的标签和图结构信息进行“标签传播”。对于OGBN-product和OGBN-arxiv，传播的信息是对应节点所属的类别；对于OGBN-proteins，我们对112维的节点标签进行逐元素的传播。
- **GCN**：使用节点本身的特征和图结构进行“卷积”操作，实现信息的聚合，最后通过线性层和softmax函数实现分类。
- **LLM**：我们本地部署了Qwen2.5 7b-Instruct模型，并使用zero-shot的方式在OGBN-arxiv的测试集上测试了基座模型的能力（15.00%）；随后，我们使用LLaMA-Factory对基座模型进行监督微调（SFT）得到了增强后的分类模型，分类准确率为23.40%（上升8.40%）。此外，我们还测试了测试时增强（Test-time scaling, TTS）策略对分类效果的影响。

由于计算资源和设备版本的限制，我们仅在MLP模型上测试了完整的OGBN数据集，对于LPA，GCN，我们随机从整图中抽出一定比例的节点并构建子图，在子图上实现了对应的算法并完成测评。完整的项目代码开源在[Ruiqi327/THU-ML-2025-Fall](#)。

二、算法与数据集介绍

基线算法

- MLP是一种基于全连接结构的传统神经网络，由输入层、隐藏层和输出层组成。核心原理是通过**层级化的线性变换与非线性激活**拟合复杂函数：输入特征经输入层传入后，每个隐藏层神经元对前一层所有节点的输出进行加权求和，再通过非线性激活函数（如ReLU、Sigmoid）引入非线性表达能力，最终由输出层输出预测结果（分类任务常用Softmax，回归任务直接输出连续值）。模型通过反向传播算法最小化损失函数，迭代更新各层权重和偏置，实现对数据模式的学习。
- GCN是专为图结构数据（节点+边）设计的深度学习模型，核心是**利用节点的邻域信息聚合特征**。其原理基于图卷积操作：对于每个节点，先通过邻接矩阵 $\mathbf{A}$ （或归一化邻接矩阵 $\tilde{\mathbf{A}} = \hat{\mathbf{D}}^{-1/2}(\mathbf{A} + \mathbf{I})\hat{\mathbf{D}}^{-1/2}$ ， $\mathbf{I}$ 为单位矩阵， $\hat{\mathbf{D}}$ 为度矩阵）捕捉其邻居节点集合，再将自身特征 $\mathbf{X}$ 与邻居特征进行加权聚合（核心公式： $\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{A}}\mathbf{H}^{(l)}\mathbf{W}^{(l)})$ ）， $\mathbf{H}^{(l)}$ 为第 l 层特征， $\mathbf{W}^{(l)}$ 为可学习权重， σ 为激活函

数)。通过多层此类操作，节点特征逐渐融合局部乃至全局的图结构信息，适用于节点分类、链路预测等图任务。

- LPA是一种无监督/半监督学习的图节点分类算法，核心是**基于“相邻节点标签相似”的假设进行标签传播**。算法无需训练参数，仅依赖图的拓扑结构：初始时，部分节点拥有真实标签，其余节点标签未知；迭代过程中，每个节点将自身标签更新为其所有邻居节点中出现频率最高的标签（或通过概率加权聚合邻居标签），直至标签分布收敛。其本质是利用图的连通性和社区结构，让标签从已知节点“扩散”到未知节点，实现高效的半监督分类。
- LLM是基于Transformer架构（尤其是自注意力机制）的大规模预训练语言模型，核心是**捕捉文本序列的长距离依赖和语言规律**。原理分为两步：1) 预训练阶段：在海量文本数据上学习语言的语法、语义和逻辑，2) 下游适配阶段：通过微调或提示工程，将预训练获得的通用语言能力迁移到具体任务（如文本分类、生成、翻译）。

OGBN数据集

- **OGBN-Product**: 该数据集源于亚马逊产品联合采购网络，适配电商场景下的商品关联分析与类别预测研究。其数据规模达约 244.9 万个节点（代表亚马逊平台的产品）和 6185.9 万条边（代表产品间的共同购买关系）。数据集提供的节点特征，是从产品描述中提取词袋特征后经主成分分析降维至 100 维得到的；同时按产品销售排名划分数据，前 8% 用于训练、后续 2% 用于验证，剩余作为测试集。它的预测任务为多类别分类，需预测产品所属的 47 个顶级类别。
- **OGBN-Proteins**: 该数据集背景是生物领域的蛋白质关联分析，聚焦不同物种蛋白质间的相互作用研究。其包含来自 8 个物种的蛋白质节点，边则代表蛋白质间具有生物学意义的关联，且所有边都带有 8 维特征，每个维度对应一种关联类型的强度，数值介于 0 - 1 之间。因未直接提供节点特征，实验中常以传入边的平均边缘特征作为节点特征，数据按蛋白质来源的物种划分训练、验证与测试集。预测任务属于多标签二元分类，需判断蛋白质是否具备 112 种特定功能，模型性能通过这 112 个任务的 ROC - AUC 分数平均值来衡量。
- **OGBN-arxiv**: 该数据集构建自 arXiv 平台的计算机科学论文引文网络。它是一个有向图，节点代表单篇 arXiv 论文，有向边代表论文间的引用关系，每篇论文都具备 128 维特征向量，该向量由标题和摘要中单词的嵌入值平均得到。数据集按论文发表年份进行分割，2017 年及以前的论文作为训练集，2018 年的作为验证集，2019 年及以后的作为测试集。其预测任务为 40 分类问题，需预测每篇论文所属的计算机科学细分领域。

三、实验设置&主要结果

1、MLP

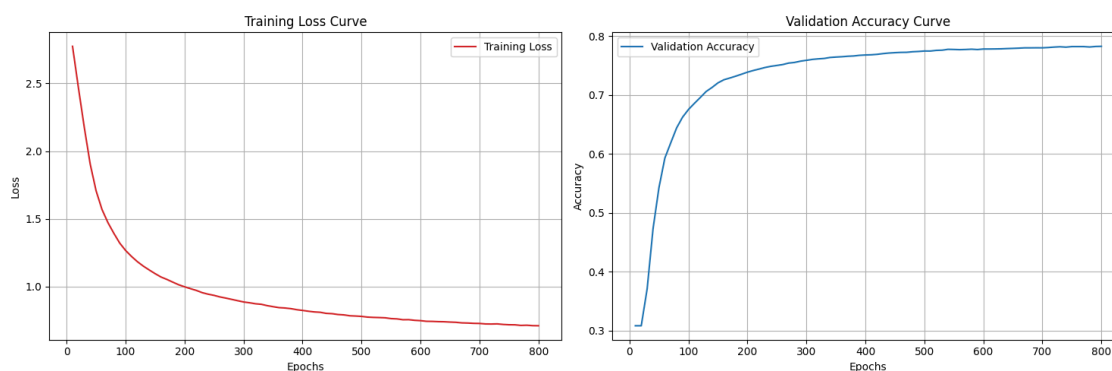
模型实现：我们使用torch.nn库来实现MLP模块：其由输入层、隐藏层和输出层组成。在隐藏层中，我们使用RELU作为激活函数；输出层则由线性层组成。实现代码如下：

```
class MLP(nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels, num_layers, dropout):
        super(MLP, self).__init__()
        self.layers = nn.ModuleList()
        self.layers.append(nn.Linear(in_channels, hidden_channels))
        for _ in range(num_layers - 2):
            self.layers.append(nn.Linear(hidden_channels, hidden_channels))
        self.layers.append(nn.Linear(hidden_channels, out_channels))
        self.dropout = dropout
```

```
def forward(self, x):
    for i, layer in enumerate(self.layers[:-1]):
        x = layer(x)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)
    x = self.layers[-1](x)
    return x
```

在训练过程中，我们将每个节点的自身特征和节点标签作为输入-输出对，不考虑整体的图特征，并使用交叉熵作为损失函数进行训练。

模型训练：我们锚定文献【1】中的实验效果来调整超参数，最终设置隐藏层层数为3，维度为512，学习率为 $1e-3$ ，迭代轮数为800，我们在三个数据集上测试了MLP的效果。训练曲线图一所示。受篇幅限制，这里我们仅展示MLP在OGBN-product上的训练曲线，全部的图片可在[Ruigi327/THU-ML-2025-Fall](#)处获得。可以看出的是，随着迭代次数的增多损失函数逐渐降低并收敛，这表明了训练的稳定性。



图一：3层512维MLP在OGBN-product上的训练Loss曲线和验证集准确率变化曲线

测试结果：取训练过程中在验证集上取得最佳表现的模型权重在测试集上进行测试。得到的结果如下表所示。括号内为文献【1】给出的标准结果。从表中可以看出，我们实现的MLP具有与标准结果相似的性能。此外，我们还可以总结出如下结论：

- MLP在OGBN-product数据集的验证集和测试集准确率差别较大。这可能与product的划分方式有关（采用销量排名来划分验证/测试集），造成了不同验证/测试集间的数据分布差异。
- MLP在OGBN-Arxiv数据集上表现一般，这可能与其特征提取结果质量较低有关。

Datasets	Validation	Test
OGBN-Product (Acc.)	0.7829 (0.7554)	0.6330 (0.6106)
OGBN-Arxiv (ROC-AUC)	0.5784 (0.5765)	0.5578 (0.5550)
OGBN-Proteins (Acc.)	0.7365 (0.7706)	0.7195 (0.7204)

2、GCN

算法实现：我们使用 torch_geometric.nn库来实现一个标准图卷积神经网络【2】。具体实现如下：

```
class GCN(torch.nn.Module):
    def __init__(self, in_channels, hidden_channels, out_channels, num_layers, dropout):
        super().__init__()
        self.convs = torch.nn.ModuleList()
```

```

self.convs.append(GCNConv(in_channels, hidden_channels))
for _ in range(num_layers - 2):
    self.convs.append(GCNConv(hidden_channels, hidden_channels))
self.convs.append(GCNConv(hidden_channels, out_channels))
self.dropout = dropout

def forward(self, x, edge_index):
    for conv in self.convs[:-1]:
        x = conv(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, p=self.dropout, training=self.training)
    x = self.convs[-1](x, edge_index)
    return x

```

由于标准的OGBN-product和OGBN-proteins数据集内包含数千万个节点和边的关系，维护一个邻接矩阵超过了本地支持的最大显存。因此，我们还额外实现了一个子图采样的功能：从原始数据集中按比例抽取出一定规模的节点，并搜索出这些节点相关的边，重新构造了一个新的、较小规模的子图。具体实现方案如下：

```

if subsample_ratio < 1.0:
    num_nodes_sample = int(num_nodes_full * subsample_ratio)
    subset = torch.randperm(num_nodes_full)[:num_nodes_sample]
    subset = torch.sort(subset).values
    edge_index, _ = subgraph(subset, full_edge_index, relabel_nodes=True, num_nodes=num_nodes_full)
    x = full_x[subset]
    y = full_y[subset]

    node_map = torch.full((num_nodes_full,), -1, dtype=torch.long)
    node_map[subset] = torch.arange(num_nodes_sample)

    split_idx = {}
    for split in ['train', 'valid', 'test']:
        mask = torch.isin(full_split_idx[split], subset)
        split_idx[split] = node_map[full_split_idx[split][mask]]

```

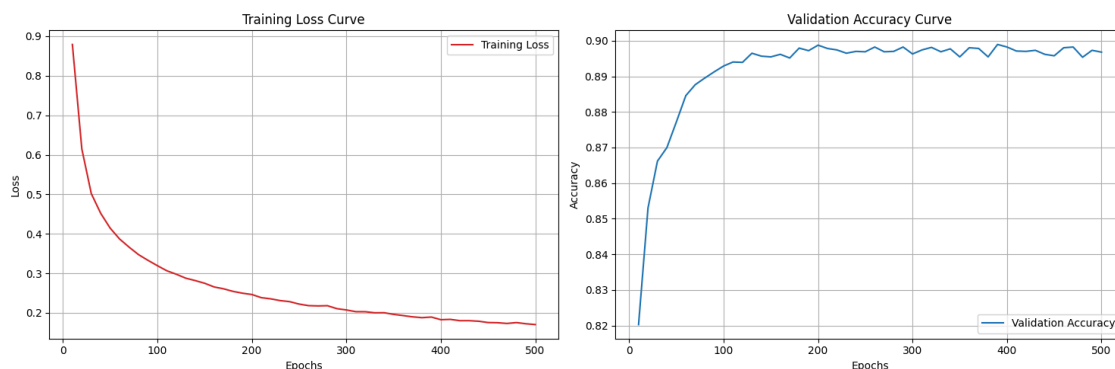
对于OGBN-arxiv数据集，我们则直接采用原始图数据集。

模型训练：在训练时，我们采用全图学习模式，即：模型可以访问整个子图的结构和所有节点的特征，但损失的计算和权重的更新只依赖于训练集节点的标签。我们为不同的训练集设置了不同的超参数，具体设置如下表所示：

Datasets	Layers (All)	Dimension	Epoch	lr	Sample
OGBN-Product	3	256	500	1e-2	0.25
OGBN-Arxiv	3	256	300	1e-2	0.2
OGBN-Proteins	3	256	800	1e-2	0.2

图二展示了单层隐藏层在OGBN-product数据集上的训练损失曲线和验证集准确率曲线。在其他数据集上的训练曲线可在[Ruqi327/THU-ML-2025-Fall](https://ruiqi327.github.io/THU-ML-2025-Fall/)处获得。可以看出的是，随着迭代次数的增多，损失和验证集

准确率都逐渐收敛。



图二：单层GCN在OGBN-product数据集上的训练损失函数和验证集准确率变化曲线

测试结果：取划分的子测试集（OGBN-product，OGBN-proteins数据集，对OGBN-arxiv数据集，测试全部的测试集），运行上述模型并观察效果。准确率如下表所示（括号内为文献【1】给出的标准结果）：

Datasets	Validation	Test
OGBN-Product (Acc.)	0.8989 (0.9200)	0.7106 (0.7564)
OGBN-Arxiv (ROC-AUC)	0.7116 (0.7300)	0.7013 (0.7174)
OGBN-Proteins (Acc.)	0.7752 (0.7921)	0.7236 (0.7251)

从表中可以观察到，相较于仅用节点单独特征的MLP，GCN在利用了图结构信息后有着更好的分类表现。同时，一些在MLP分类中观察到的问题，如模型在OGBN-product的验证和测试集上效果差别较大的现象依然出现，证明了数据集本身存在的一些数据分布不均的问题。

3、LPA

算法实现：我们不采用现有的torch_geometric，而手动编写了LPA的运行逻辑【3】，其包括两个阶段：

- 初始化：为所有训练集中的节点赋予其真实的标签，为所有其他节点（验证集和测试集）赋予一个唯一的初始标签。
- 迭代传播：在每一轮迭代中，随机打乱所有节点的顺序。遍历每一个节点（训练集节点除外），查看其所有邻居节点的当前标签，将该节点的标签更新为其邻居中最常出现的那个标签（“少数服从多数”）。

具体实现如下：

```
@torch.no_grad()
def run_lpa(adj, y_true, train_idx, valid_idx, num_nodes, max_iter, device):
    print("\n--- 开始执行标签传播算法 (LPA) ---")
    y_pred = torch.arange(num_nodes, dtype=torch.long)
    y_pred[train_idx] = y_true[train_idx]
    y_pred = y_pred.to(device)
    y_true_device = y_true.to(device)
    validation_history = []
    for node, neighbors in adj.items():
        adj[node] = neighbors.to(device)
    for i in range(max_iter):
        changed = False
        nodes_to_update = np.random.permutation(num_nodes)
```

```

pbar = tqdm(nodes_to_update, desc=f"LPA Iteration {i+1}/{max_iter}", leave=False)
for node in pbar:
    if node in train_idx:
        continue
    neighbors = adj.get(node)
    if neighbors is None or len(neighbors) == 0:
        continue
    neighbor_labels = y_pred[neighbors]
    if len(neighbor_labels) == 0: continue
    counts = torch.bincount(neighbor_labels)
    new_label = torch.argmax(counts)
    if y_pred[node] != new_label:
        y_pred[node] = new_label
        changed = True

```

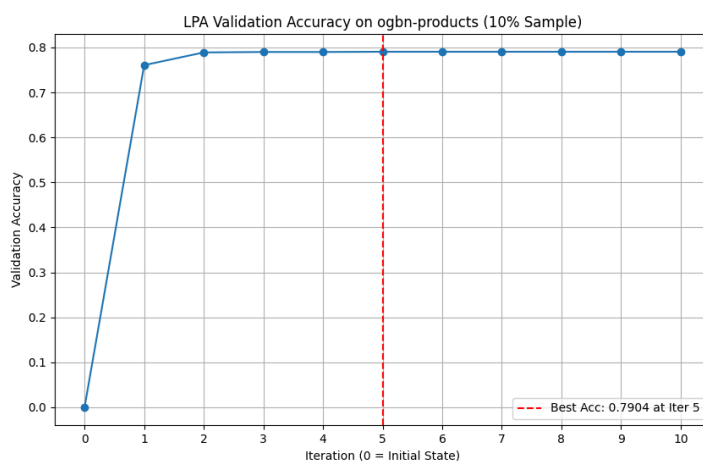
对于OGBN-product和OGBN-proteins数据集，我们与GCN的实现保持一致，只采用一部分子图来运行LPA算法，而对于OGBN-arxiv数据集，我们则在整个图上运行。特别的，由于OGBN-proteins数据集的每个标签是一个112维的向量，我们实现了逐元素的标签传播。具体实现代码如下：

```

neighbor_labels = labels[train_neighbors]
label_counts = torch.sum(neighbor_labels, dim=0)
threshold = len(train_neighbors) / 2.0
new_label_vector = (label_counts > threshold).float()

```

模型训练：LPA算法本质上是一个无监督算法，不需要额外设置超参数，也不存在训练损失曲线。设置迭代次数为10，我们在三个数据集上运行了实现的算法。实验结果如图三所示。篇幅所限，这里我们仅展示LPA算法在OGBN-products数据集上的验证集准确率变化曲线。可以看出的是，在三次迭代后LPA算法便趋近于收敛，并在第五次迭代时取得了最大准确率。



图三：LPA算法在OGBN-product数据集上的验证集准确率变化曲线

测试结果：我们在划分的子图测试集或完整测试集上测试了LPA算法的性能，测试结果如下表所示：

Datasets	Validation	Test
OGBN-Product (Acc.)	0.7904	0.4676

OGBN-Arxiv (ROC-AUC)	0.7915	0.4553
OGBN-Proteins (Acc.)	0.6481	0.6431

对比MLP和GCN算法，LPA算法取得效果较差。我们认为这是因为LPA算法作为一种无监督的算法，本质上只是基于“邻近样本特征相似”这一朴素的想法，并不能有效地提取出不同节点的特征表示。同样，LPA算法也表现出在OGBN-product和OGBN-proteins数据集上，验证集效果远远优于测试集效果的现象。

4、LLM

算法实现：我们使用Qwen2.5-7B-Instruct作为基座模型，将OGBN-arxiv数据集中对应的文章标题和摘要作为输入，要求模型在zero-shot的形式预测该文章在arxiv格式下的顶级类别。为了方便模型理解分类类别的语义，我们不采用顶级类别的缩写形式，如cs.ml等，而将其转义为具体的类别如machine learning。我们发现只需要提示模型“使用arxiv标准分类来预测”，就可以让模型输出预期的类别，不需要额外的提示补充。具体来说，我们实现了以下三个算法：

- 我们测试了基座模型Qwen2.5-7B-Instruct的准确率作为基线指标。
- 我们使用抽取的8000条数据作为训练集，使用LLaMA-Factory框架对基座模型进行lora低秩微调，测试训练后模型的效果。
- 我们对训练后的模型做测试时扩展（Test-time scaling），为每个prompt生成8个回答，并采用众数投票的方式进一步提升模型的分类性能。

在使用LLaMA-Factory进行lora训练时，我们在.yaml文件中设置关键参数如下：

```
lora_rank: 8
lora_target: all
template: qwen
cutoff_len: 1024
max_samples: 8000
preprocessing_num_workers: 16

per_device_train_batch_size: 1
gradient_accumulation_steps: 8
learning_rate: 1e-5
num_train_epochs: 3.0
lr_scheduler_type: cosine
warmup_ratio: 0.1
bf16: true
```

在推理时，我们使用vllm框架动态加载lora模块，并实现加速推理。同时，我们设置temperature为0.9，最大输出长度为512，并使用如下的prompt：

```
Classify the following research paper into its appropriate category based on the title
and abstract. The category name follows from standard arxiv disciplines (full name, no
abbreviation. For example, Machine Learning, Computational Linguistics, etc). You
should put the category in the block <answer></answer>.
```

实验结果：我们在筛选出的500条训练数据上运行了测试了上述模型。同时，我们还保存了基座模型在训练集上训练多个epoch后的检查点（checkpoint），并同样完成了测评。测试结果如下表所示。

Models	无TTS	有TTS
--------	------	------

Qwen2.5-7B-Instruct	15.0%	15.6%
Qwen2.5-7B-Instruct+SFT (0.5 epoch)	23.4%	23.8%
Qwen2.5-7B-Instruct+SFT (1 epoch)	9.8%	10.0%
Qwen2.5-7B-Instruct+SFT (2 epoch)	5.4%	5.4%

从表中可以看出，TTS策略和微调策略对提升LLM分类性能都有一定的效果。然而，当微调轮数超过0.5时，模型却出现了过拟合现象：训练loss持续下降但在测试集上的效果却一直下跌。我们猜测只采用分类类别作为单独的监督信号不足以让模型学习到摘要-类别的内在关联模式。

Case Study: 为了探索训练后模型出现灾难性遗忘的回答形式，我们提取了部分模型的回答。我们发现当训练超过1个epoch时，模型存在以下两种错误范式：

- 只回答<answer>Information Theory</answer>
- 在回答中出现多个<answer></answer>块

我们猜测这可能与提取出的训练集数据中存在着较多的Information Theory标签导致。同时，由于单一文章的内容往往是多个学科的交叉，模型有可能将其认作多个领域并给出多个<answer></answer>块。例如：

```
<answer>Computer Vision and Pattern Recognition</answer> The title and abstract suggest that the research is focused on conversational QA and reasoning, which falls under the broader category of Computer Vision and Pattern Recognition. This category is appropriate as it encompasses the use of neural networks and pattern recognition techniques in natural language processing tasks. However, it's worth noting that this paper could also fit into other categories such as Machine Learning or Artificial Intelligence, but Computer Vision and Pattern Recognition is the most specific and relevant based on the provided information. <answer>Machine Learning</answer> would also be a suitable choice. Given the options and the specific focus on conversational QA and reasoning, Computer Vision and Pattern Recognition is the most fitting category. <answer>Machine Learning</answer> would also be a valid choice, but Computer Vision and Pattern Recognition is more specific to the content described. <answer>Machine Learning</answer> is the most appropriate choice given the standard arxiv disciplines. <answer>Machine Learning</answer> is the most appropriate choice given the standard arxiv disciplines.
```

四、总结

在本实验中，我们在OGBN的三个数据集上实现了MLP，GCN，LPA和LLM四种基线算法。我们发现：1) 引入图结构信息对提升模型分类能力有着显著的提升，2) OGBN-product数据集的训练/验证/测试集间存在着较大的数据分布差异，导致模型表现差别较大，3) 在面对超大规模的图数据集时，往往需要较大的硬件资源支持，需要考虑子图抽取或者使用小批量迭代的算法。4) 尽管可以通过微调的方式提高分类性能，现有大模型在处理OGBN-arxiv的分类任务上仍有较大不足。

References

- 【1】 Hu, W., Fey, M., et al. (2020). Open Graph Benchmark: Datasets for Machine Learning on Graphs
- 【2】 Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks.
- 【3】 Zhu, X., & Ghahramani, Z. (2002). Learning from labeled and unlabeled data with label propagation.